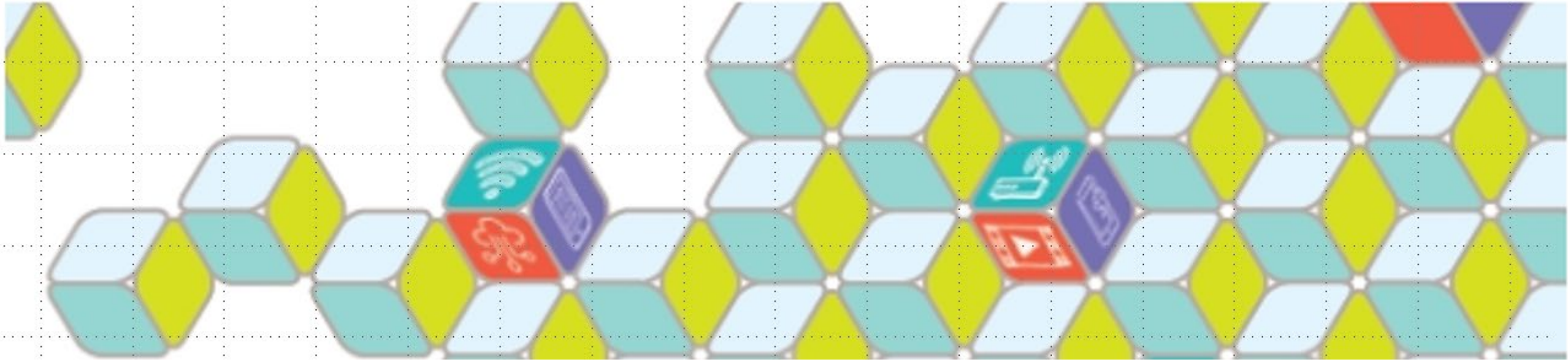


Programming Method
(using C, C++, for Games)

Prof. Tae Soo Yun
Dept. of Game, International College, Dongseo University

DSU Dongseo University 동서대학교

Ch05_Pointers and Arrays

The Goal of the ch. 05

A pointer is a variable that contains the address of a variable.

To learn about pointer and indirect addressing, with arrays and file.

To see how to access external data files in a program and to be able to read from input files and write to output files using file pointers.

Chapter 5.	**Pointers and Arrays**	**93**
5.1	Pointers and Addresses	93
5.2	Pointers and Function Arguments	95
5.3	Pointers and Arrays	97
5.4	Address Arithmetic	100
5.5	Character Pointers and Functions	104
5.6	Pointer Arrays; Pointers to Pointers	107
5.7	Multi-dimensional Arrays	110
5.8	Initialization of Pointer Arrays	113
5.9	Pointers vs. Multi-dimensional Arrays	113
5.10	Command-line Arguments	114
5.11	Pointers to Functions	118
5.12	Complicated Declarations	122

5.1 Pointers and Addresses

❖ A pointer is a variable that contains the address of a variable.

- ✓ syntax : `type *variable ;`
- ✓ ex : `float *p;`
- ✓ Declare p as a pointer variable of type "pointer to float", that is, the value of the pointer variable p is a memory address to be stored float value.

❖ unary op. & gives the address of an object. p = &c;

- ✓ Assign the address of c to the variable p, "point to" c.

```
int x =1, y = 2, z[10];
int *ip;                    // ip is a pointer to int

ip = &x;                    // ip now points to x
y = *ip;                    // y is now 1 ( *ip : dereferencing)
*ip = 0 ;                   // x is now 0
ip = &z[0];                 // ip now points to z[0]
```

❖Usage in function declaration

- ✓ double *dp, atof(char *); *dp have value of type double , the argument of atof is a pointer to char.

❖ increment /decrement the content of p (pointer) points at

- ✓ *ip = *ip + 10;
- ✓ y = *ip + 1;
- ✓ *ip += 1, ++*ip, (*ip)++ : the same result, but *ip++ (unary operator *, ++ associate right to left)

❖Exercise) trace the execution of the following fragement.

```
Int m = 10, n = 5;
Int *mp, *np;
mp = &m; np = &np;
*mp = *mp + *np;
*np = *mp - *np;
Printf(%d %d \n%d %d\n", m, *mp, n, *np);
```

5.2 Pointers and Function Arguments

❖ call by value vs, call by reference

- ✓ In call by value, no direct way to alter a variable
- ✓ Suppose that we write a program to exchange two values in a sorting routine.

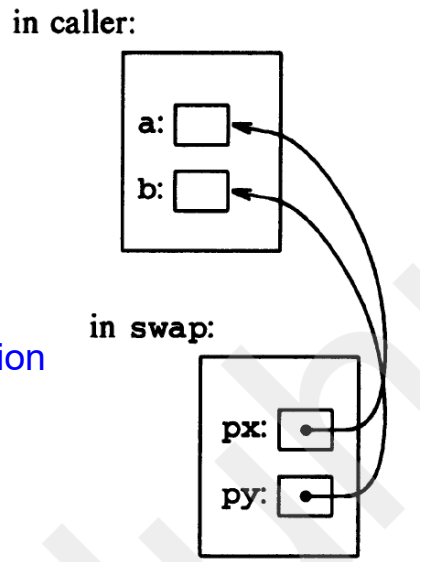
```
void swap(int x, int y)    // wrong
{
    int temp;

    temp = x;
    x = y;
    y = temp;
}
```

```
void swap(int *px, int *py)          // interchange *px and *py
{
    int temp;

    temp = *px;
    *px = *py;
    *py = temp;
}
```

```
swap(&a, &b)          // calling function
```



5.2 Pointers and Function Arguments

❖ write a program getint that performs free-format input conversion by breaking a stream of characters into integer values, one integer per call.

```
int main(void) {
    int n, s, array[SIZE], getint(int*);

    for (n = 0; n < SIZE && getint(&array[n]) != EOF;) {
        /* For debug purposes */
        /* ( (res!=0) ? n++:0 ) will add only valid values to the array */
        printf("storing in n = %d, getint %d₩n", n, array[n]);
    }

    printf("storing in n = %d, getint %d₩n", n, array[n]);

    for (s = 0; s <= n; s++)
        printf("%d", array[s]);

    return 0;
}
```

```
#include <ctype.h>

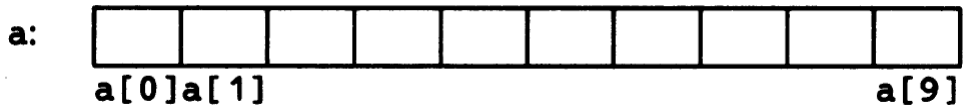
int getch(void);
void ungetch(int);

/* getint:  get next integer from input into *pn */
int getint(int *pn)
{
    int c, sign;

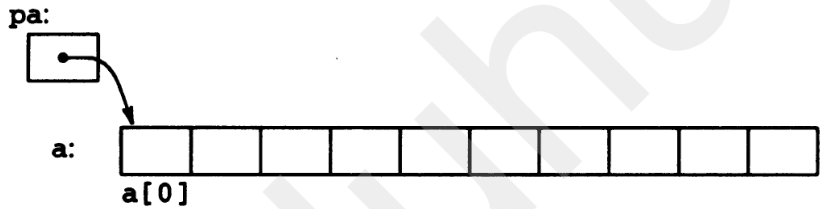
    while (isspace(c = getch()))    /* skip white space */
        ;
    if (!isdigit(c) && c != EOF && c != '+' && c != '-') {
        ungetch(c);    /* it's not a number */
        return 0;
    }
    sign = (c == '-') ? -1 : 1;
    if (c == '+' || c == '-')
        c = getch();
    for (*pn = 0; isdigit(c); c = getch())
        *pn = 10 * *pn + (c - '0');
    *pn *= sign;
    if (c != EOF)
        ungetch(c);
    return c;
}
```

5.3 Pointers and Arrarys

- in C, there is a strong relationship between pointers and arrays
  - ✓ Any operation that can be achieved by array subscripting can also be done with pointers.
- declaration of array and managing by pointer.
  - ✓ int a[10]; defines an array **a** of size 10, that is, a block of 10 consecutive objects named a[0], a[1], ···, a[9]

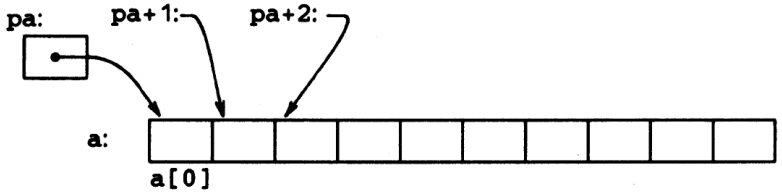


  - ✓ `int *pa;` `pa = &a[0];` sets pa to point to element zero of a; that is, pa contains the address of a[0].



  - ✓ x = *pa; copy contents of a[0] into x. : dereferencing

(pa+1) refers to the contents of a[1], ie. a[i],(pa+i) are the same.



5.4 Character Pointers and Functions

❖ Difference between array name and a pointer

- ✓ A pointer is a variable, so pa =a, pa++ are legal
- ✓ Array name is not variable, a =pa, a++ are illegal.

❖ when an array name is passed to a function.

- ✓ What passed is the address of the first element
- ✓ This argument is a local variable, array name argument is pointer
- ✓ Can calls like

```
strlen("hello, world");   /* string constant */
strlen(array);            /* char array[100]; */
strlen(ptr);              /* char *ptr; */
```

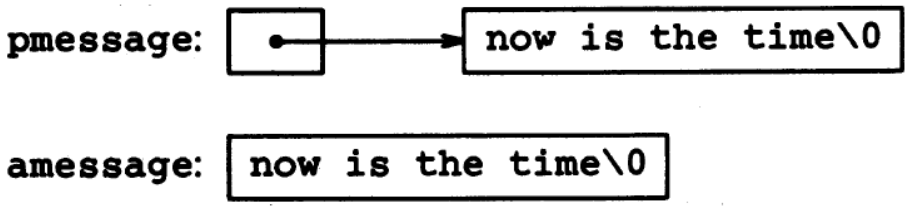
- ✓ char s[] and char * are equivalent

```
/* strlen:  return length of string s */
int strlen(char *s)
{
    int n;

    for (n = 0; *s != '\0'; s++)
        n++;
    return n;
}
```

5.4 Character Pointers and Functions

- char amessage[] = “now is the time” ; // an array
- char *pmessage = “now is the time”; // a pointer




```
/* strcpy:  copy t to s; array subscript version */
void strcpy(char *s, char *t)
{
    int i;

    i = 0;
    while ((s[i] = t[i]) != '\0')
        i++;
}
```

```
/* strcpy:  copy t to s; pointer version 2 */
void strcpy(char *s, char *t)
{
    while ((*s++ = *t++) != '\0')
        ;
}
```

```
/* strcpy:  copy t to s; pointer version 1 */
void strcpy(char *s, char *t)
{
    while ((*s = *t) != '\0') {
        s++;
        t++;
    }
}
```

```
/* strcpy:  copy t to s; pointer version 3 */
void strcpy(char *s, char *t)
{
    while (*s++ = *t++)
        ;
}
```

5.4 Character Pointers and Functions

- strcmp(s,t) which compares the character strings s and t, and returns negative, zero, or positive if s is alphabetically less than, equal to, or greater that t.
  - ✓ Header ⟨string.h⟩ contains declarations for these functions related to string.

```
/* strcmp:  return <0 if s<t, 0 if s==t, >0 if s>t */
int strcmp(char *s, char *t)
{
    int i;

    for (i = 0; s[i] == t[i]; i++)
        if (s[i] == '\0')
            return 0;
    return s[i] - t[i];
}
```

```
/* strcmp:  return <0 if s<t, 0 if s==t, >0 if s>t */
int strcmp(char *s, char *t)
{
    for ( ; *s == *t; s++, t++)
        if (*s == '\0')
            return 0;
    return *s - *t;
}
```

Exercise 5-3. Write a pointer version of the function `strcat` that we showed in Chapter 2: `strcat(s,t)` copies the string `t` to the end of `s`. □

Exercise 5-4. Write the function `strend(s,t)`, which returns 1 if the string `t` occurs at the end of the string `s`, and zero otherwise. □

Exercise 5-5. Write versions of the library functions `strncpy`, `strncat`, and `strncmp`, which operate on at most the first `n` characters of their argument strings. For example, `strncpy(s,t,n)` copies at most `n` characters of `t` to `s`. Full descriptions are in Appendix B. □

Exercise 5-6. Rewrite appropriate programs from earlier chapters and exercises with pointers instead of array indexing. Good possibilities include `getline` (Chapters 1 and 4), `atoi`, `itoa`, and their variants (Chapters 2, 3, and 4), `reverse` (Chapter 3), and `strindex` and `getop` (Chapter 4). □

❖For the purpose of exercise, use the main routine like this.

```c
#include<stdio.h>
#define MAXLINE 1000

int mgetline(char line[], int lim);
void mystrcat(char*, char*);

int main(void) {
    int len;
    char s[MAXLINE], t[MAXLINE];

    putchar('s');    putchar(':');
    mgetline(s, MAXLINE);

    putchar('t');    putchar(':');
    mgetline(t, MAXLINE);

    mystrcat(s, t);
    printf("%s", s);
    return 0;
}
```

```c
#include<stdio.h>
#define MAXLINE 1000

int mgetline(char s[], int lim);
int strend(char* s, char* t);
int mystrlen(char* t);

int main(void) {
    char s[MAXLINE], t[MAXLINE];
    int ret;

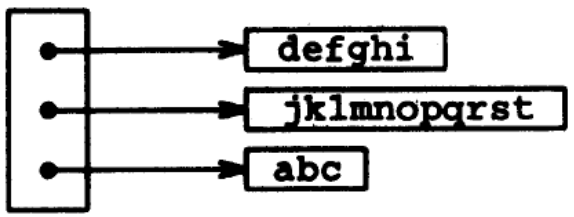
    putchar('s');    putchar(':');
    mgetline(s, MAXLINE);

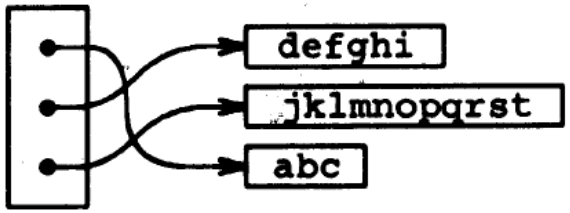
    putchar('t');    putchar(':');
    mgetline(t, MAXLINE);

    ret = strend(s, t);
    printf("%d", ret);
    return 0;
}
```

5.6 Pointer Arrays : Pointers to Pointers

- Since pointers are variables, they can be stored in arrays just as other variables can.
  - ✓ That is, we can consider pointer as an array name and, can use the pointer as an array
- Write a program that will sort a set of text lines into alphabetic order. We will use the getline functions in previous class.
  - ✓ We need a data representation that will cope efficiently and conveniently with variable-length text lines.
  - ✓ This is where the array of pointer enters.
  - ✓ Declaration : char *lineptr[MAXLINES] ;
    - lineptr is an array of MAXLINES elements, each element of which is a pointer to a char.
    - So, lineptr[i] is a character pointer, *lineptr[i] is the first character of i-th textline
  - ✓ Figure) The exchange process of two out-of-order lines,
    - Just exchange the pointers in the pointer array, not the text lines





❖Download link

```c
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>
#include<string.h>

#define MAXLINES 5000   /* max #lines to be sorted */

char* lineptr[MAXLINES];

int readlines(char* lineptr[], int maxlines);
void writelines(char* lineptr[], int nlines);
void qsort(char* v[], int left, int right);

/* sort input lines */
int main(void) {
    int nlines; /* number of input lines read */

    if ((nlines = readlines(lineptr, MAXLINES)) >= 0) {
        qsort(lineptr, 0, nlines - 1);
        writelines(lineptr, nlines);
        return 0;
    }
    else {
        printf("error: input too big to sort ₩n");
        return 1;
    }
}
```

```c
#define MAXLEN 1000 /* max length of any input line */
#define MAXSTOR 5000

int mgetline(char*, int);
char* alloc(int);

/* readlines: read input lines */
int readlines(char* lineptr[], int maxlines) {
    int len, nlines;
    char *p, line[MAXLEN];

    nlines = 0;
    while ((len = mgetline(line, MAXLEN)) > 0)
        if (nlines >= maxlines || (p=alloc(len)) == NULL)
            return -1;
        else {
            line[len - 1] = '₩0';
            strcpy(p, line);
            lineptr[nlines++] = p;
        }
    return nlines;
}

/* writelines: write output lines */
void writelines(char* lineptr[], int nlines) {
    int i;
    for (i = 0; i < nlines; i++)
        printf("%s₩n", lineptr[i]);
}
```

```c
/* qsort: sort v[left] ... v[right] into increasing order */
void qsort(char* v[], int left, int right) {
    int i, last;
    void swap(char* v[], int i, int j);

    if (left >= right)
        return;
    swap(v, left, (left + right) / 2);

    last = left;

    for (i = left + 1; i <= right; i++)
        if (strcmp(v[i], v[left]) < 0)
            swap(v, ++last, i);
    swap(v, left, last);
    qsort(v, left, last - 1);
    qsort(v, last + 1, right);
}

/* swap: interchange v[i] and v[j] */

void swap(char* v[], int i, int j) {
    char* temp;

    temp = v[i];
    v[i] = v[j];
    v[j] = temp;
}
```

```c
#define ALLOCSIZE 10000  /* size of available space */

static char allocbuf[ALLOCSIZE];    /* storage for alloc */
static char* allocp = allocbuf;     /* next free position */

char* alloc(int n)  /* return pointer to n characters */
{
    if (allocbuf + ALLOCSIZE - allocp >= n) {
        allocp += n;
        return allocp - n;
    }
    else
        return 0;
}

int mgetline(char* s, int lim) {
    int c;
    char* t = s;

    while (--lim > 0 && (c = getchar()) != EOF && c != '₩n')
        *s++ = c;
    if (c == '₩n')
        *s++ = c;

    *s = '₩0';

    return s - t;
}
```

Static memory allocation

❖alloc, and afree are implemented for static memory allocation

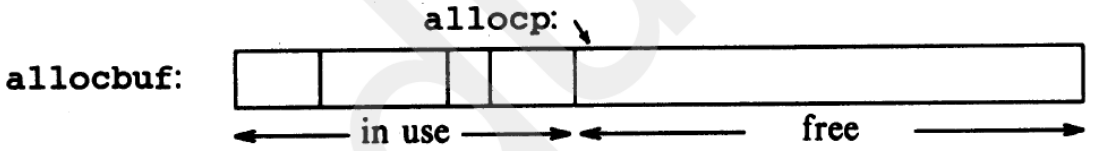
- ✓ allocbuf[] ; storage for alloc
- ✓ allocp ; points to the next free element
- ✓ alloc(n) returns the current value of allocp

```
#define ALLOCSIZE 10000 /* size of available space */

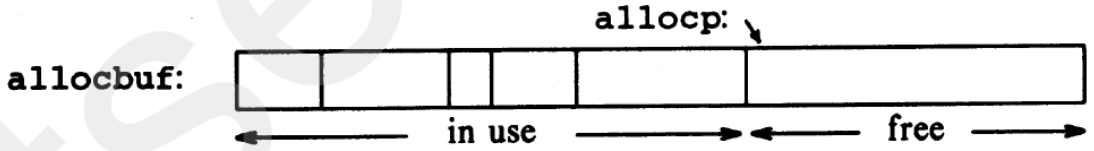
static char allocbuf[ALLOCSIZE];  /* storage for alloc */
static char *allocp = allocbuf;   /* next free position */

char *alloc(int n)  /* return pointer to n characters */
{
    if (allocbuf + ALLOCSIZE - allocp >= n) { /* it fits */
        allocp += n;
        return allocp - n; /* old p */
    } else       /* not enough room */
        return 0;
```

before call to alloc:



after call to alloc:




```
void afree(char *p)  /* free storage pointed to by p */
{
    if (p >= allocbuf && p < allocbuf + ALLOCSIZE)
        allocp = p;
}
```

Dynamic memory allocation

- Dynamic memory allocation, assign memory block to heap memory on runtime and deallocate it when it to be finished.
  - ✓ Possible to create linked list, dynamically-sized array&strings, tree, hashtables.
- Memory allocation function
  - ✓ malloc(size_t size), calloc(size_t count, size_t size), free(p); defined in stdlib.h
  - ✓ Difference : calloc initialize memory to zero, malloc not
  - ✓ malloc(n) calloc(), returns a pointer p to n-consecutive character positions.

```c
int	main(void)
{
	int *arr_1;
	int *arr_2;

	arr_1 = (int*)malloc(sizeof(int) * 4);
	arr_2 = (int*)calloc(sizeof(int) * 4, sizeof(int) * 4);

	for (int i = 0; i < 4; i++)
	{
		printf("arr_1 = %-10d\tarr_2 = %-10d\n", arr_1[i], arr_2[i]);
	}

	free(arr_1);
	free(arr_2);
}
```

```
arr_1 = 0           arr_2 = 0
arr_1 = 1610612736  arr_2 = 0
arr_1 = 0           arr_2 = 0
arr_1 = 1610612736  arr_2 = 0
```

```c
int	main(void)
{
	char	*arr;
	int	num;

	num = 3;
	arr = (char*)malloc(num);
	for (int i = 0; i < 3; i++)
	{
		arr[i] = '0' + i;
		printf("arr[%d] = %c\n", i, arr[i]);
	}
	free(arr);
}
```

```
arr[0] = 0
arr[1] = 1
arr[2] = 2
```

exercise

Exercise 5-7. Rewrite `readlines` to store lines in an array supplied by `main`, rather than calling `alloc` to maintain storage. How much faster is the program? □

```
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>
#include<string.h>

#define MAXLINES 5000   /* max #lines to be sorted */

char* lineptr[MAXLINES];
char linestor[MAXLINES];

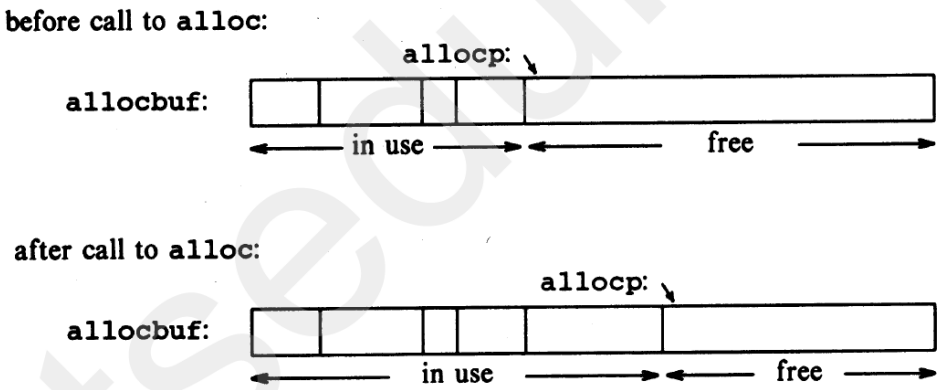
int readlines(char* lineptr[], char* linestor, int maxlines);
void writelines(char* lineptr[], int nlines);
void qsort(char* v[], int left, int right);

/* sort input lines */

int main(void) {
    int nlines; /* number of input lines read */

    if ((nlines = readlines(lineptr, linestor, MAXLINES)) >= 0) {
        qsort(lineptr, 0, nlines - 1);
        writelines(lineptr, nlines);
        return 0;
    }
    else {
        printf("error: input too big to sort \n");
        return 1;
    }
}
```

- declare linestor[maxline] instead of allocbuf
- what data type control the position instead of allocp



5.7 Multi-dimensional Arrays

- C provides rectangular multi-dimensional arrays.
- consider the problem of date conversion, from the day of the month to the day of the year and vice versa.
  - ✓ Ex) month_day(1988, 60, &m, &d) : set m to 2 and d to 29(Feb. 29th)
  - ✓ day_of_year(1988, 2,29) return 60;

  - ✓ Need some information on the # of days in Feb. according to the leap year or not
  - ✓ Or, a table of the # of days in each month.
    - static char daytab[2] [13] = {
      { 0,31,28,31,30,31,30,31,31,30, 31,30,31},
      { 0,31,29,31,30,31,30,31,31,30, 31,30,31}
      }
    - we can access elements using : daytab[i] [j], where i =0~1, j=1~12 not 0~11

- download link
- a year is a leap year if it is divisible by 4, and not by 100, except when it is divisible by 400.

```
static char daytab[2][13] = {
    {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},
    {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}
};

/* day_of_year:  set day of year from month & day */
int day_of_year(int year, int month, int day)
{
    int i, leap;

    leap = year%4 == 0 && year%100 != 0 || year%400 == 0;
    for (i = 1; i < month; i++)
        day += daytab[leap][i];
    return day;
}

  /* month_day:  set month, day from day of year */
  void month_day(int year, int yearday, int *pmonth, int *pday)
  {
      int i, leap;

      leap = year%4 == 0 && year%100 != 0 || year%400 == 0;
      for (i = 1; yearday > daytab[leap][i]; i++)
          yearday -= daytab[leap][i];
      *pmonth = i;
      *pday = yearday;
  }
```

❖Ex5-8. There is no error checking in day_of_year or month_day. Remedy this defect.

✓ What if we test the following main();

```
int main(void) {
    int yearday, month, day;

    yearday = day_of_year(1988, 2, 29);
    printf("%d₩n", yearday);

    yearday = day_of_year(2020, 2, 31);
    printf("%d₩n", yearday);

    month_day(1988, 60, &month, &day);
    printf("Month: %d, Day: %d₩n", month, day);

    month_day(1999, 366, &month, &day);
    printf("Month: %d, Day: %d₩n", month, day);

    return 0;
}
```

5.8~5.9 Initialization of Pointer Arrays

❖ Consider the problem of writing a function month_name(n), which returns a pointer to a character string containing the name of the n-th month.

- ✓ How to initialize the array of characters.
- ✓ A list of character strings are assigned to each corresponding position in the array.
- ✓ name[i] is a pointer to points i-th string

```c
/* Initialization of pointer arrays*/
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

char* month_name(int n);

/* month_name : return name of n-th month*/
char *month_name(int n)
{
    static char* name[] = {
        "illegal month",
        "January", "February", "March",
        "April", "May", "June",
        "July", "August", "September",
        "October", "November", "December"
    };
    return (n < 1 || n > 12) ? name[0] : name[n];
}
```

```c
int main(void) {

    char *s;
    int n;

    printf("input number (1-12)₩n");
    scanf("%d", &n);
    s = month_name(n);
    printf("Month name is : %s₩n ", s);
    //printf("Month name is : %s₩n ", month_name(n) );
    return 0;

}
```

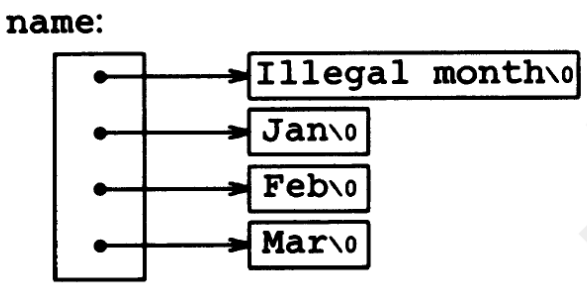
❖ difference between a 2-dimensional array and an array of pointers.

✓ Ex) int a[10] [20]; // 200 int sized locations have been set aside

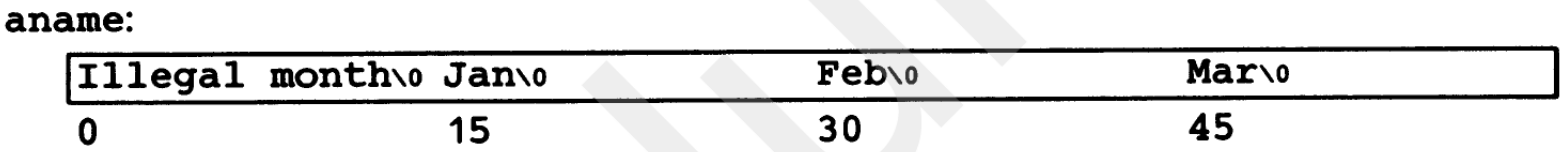
int *b[10]; // 10 cells for the pointers + the size of element that b point at

✓ pointer array : the rows of the array may be different lengths

```
char *name[] = { "Illegal month", "Jan, "Feb", "Mar" };
```




```
char aname[][15] = { "Illegal month", "Jan", "Feb", "Mar" };
```



5.10 Command-line Arguments

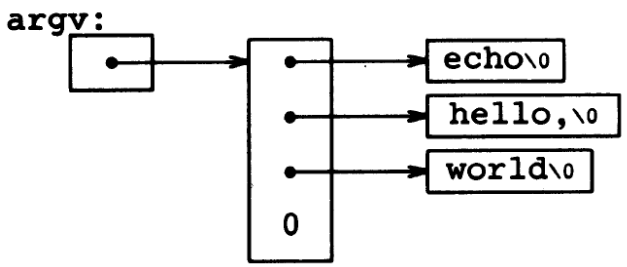
❖ Is a way to pass command-line arguments or parameters to a program when it begins executing.

- ✓ int main(void) → int main(argc, *argv[])
- ✓ ex) echo[.exe] hello, world // to print some information on console
  grade[.exe] score.txt, grade.txt // read score file and write grade file with console input
- ✓ Main() has two arguments ; first (argc) is the # of cmdline arguments, second (argv) is a pointer to an array of character string that contain the argument.
- ✓ ex) in echo.exe hello, world argc = 3, *argv[] = {"echo", "hello", "world" }

```c
#include <stdio.h>

/* echo command-line arguments ; 1st version */
int main(int argc, char* argv[])
{
    int i;

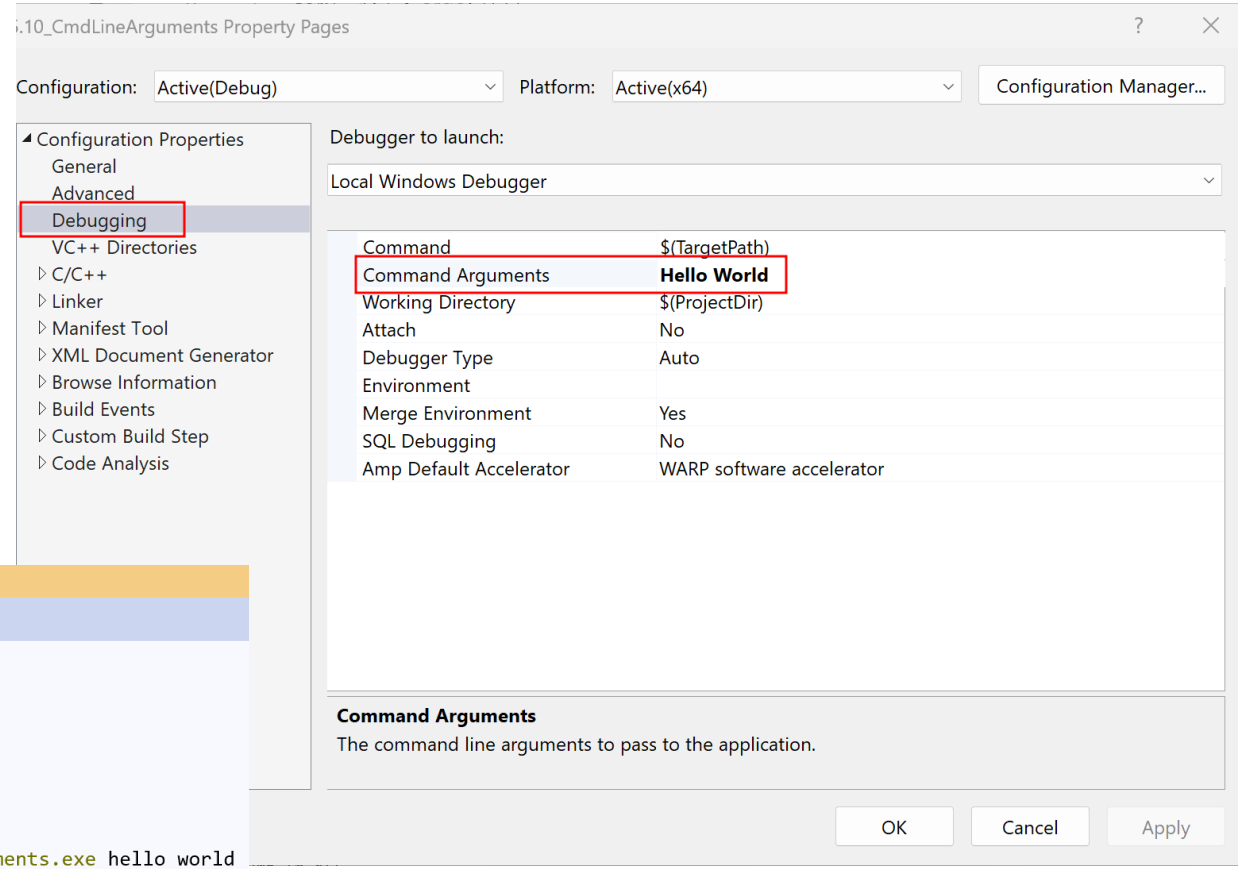
    for(i =1; i < argc; i++)
        printf("%s%s", argv[i], (i < argc - 1) ? " " : " " );
    printf("₩n");
    return 0;
}
```

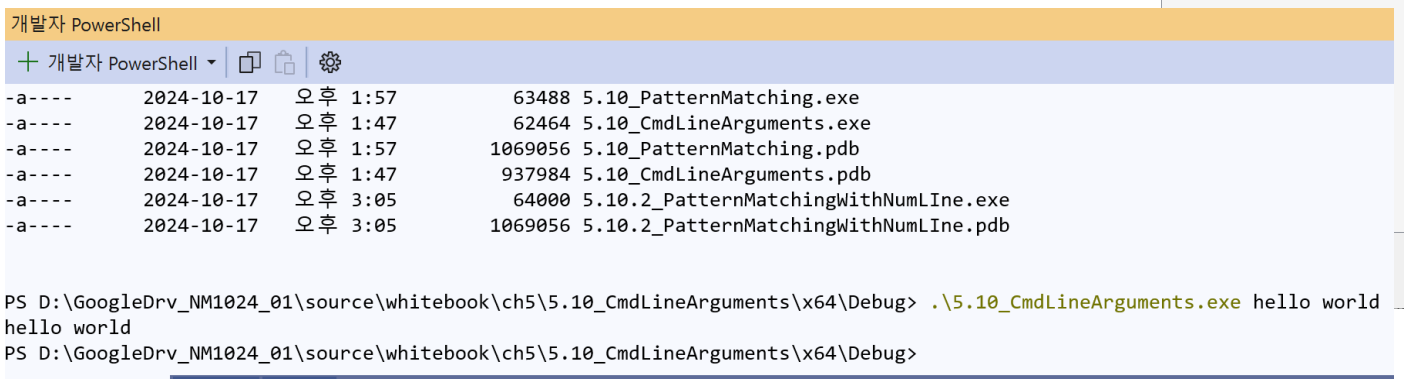


Add C/C++ Command Arguments [Visual Studio]

❖ Project > Properties > Configuration Properties > Debugging > Command Arguments

❖Powershell> View > Terminal





5.10 Command-line Arguments

❖ let us make some enhancements to the pattern-finding program from sec4.1

- ✓ int getline(char *line, int max);
- ✓ char pattern[] = "ould" // delete
- ✓ strindex(line, pattern) -> strstr(s,t) in string.h

```c
#include <stdio.h>
#include <string.h>

#define MAXLINE 1000    /* maximum input line length */

//int getline(char line[], int max);
int getline(char *line, int max);
int strindex(char source[], char searchfor[]);

//char pattern[] = "ould";  /* pattern to search for*/

/*find all lines matching pattern*/
int main(int argc, char*argv[]) {

    char line[MAXLINE];        /* current input line */
    int found = 0;

    if (argc != 2)
        printf("Usage : find pattern₩n");
    else
        while (getline(line, MAXLINE) > 0) {
//            if (strindex(line, argv[1]) != 0) { // // using strstr(s,t) in string.h
            if(strstr(line, argv[1]) != NULL){
                printf("%s", line);
                found++;
            }
        }
    return found;
}
```

- a Program that print all lines except those that match the pattern, and precede each printed line by its line number.
  - ✓ Command : Find -x -n pattern.

```
D:\GoogleDrv_NM1024_01\source\whitebook\ch5\5.10_CmdLineArguments\x64\Debug\5.10.2_PatternMatchingV
ould djfkdjfs
dkjfkde
2:dkjfkde
jdkfjdksa
3:jdkfjdksa
djfkdsjlflsaf
4:djfkdsjlflsaf
ould
dfjdks
6:dfjdks
skskskf dkfjskfjljfeijfekfjaslf
7:skskskf dkfjskfjljfeijfekfjaslf
would you like?
dirkfsfaf
9:dirkfsfaf
```

```c
#include <stdio.h>
#include <string.h>
#define MAXLINE 1000    /* maximum input line length */

int getline(char *line, int max);

/*find all lines matching pattern*/
int main(int argc, char* argv[]) {

    char line[MAXLINE];        /* current input line */
    int lineno = 0;
    int c, except = 0, number = 0, found = 0;

    while(--argc > 0 && (*++argv)[0] == '-')
        while( c=*++argv[0])
            switch (c) {
            case 'x' :
                except = 1;
                break;
            case 'n':
                number = 1;
                break;
            default:
                printf("find : illegal option %d₩n", c);
                argc = 0;
                found = -1;
                break;
            }

    if (argc != 1)
        printf("Usage : find -x -n pattern₩n");
    else
        while (getline(line, MAXLINE) > 0) {
            lineno++;
            if ((strstr(line, *argv) != NULL) != except) {
                if (number)
                    printf("%ld:", lineno);
                printf("%s", line);
                found++;
            }
        }
    return found;
}
```

Exercise 5-10. Write the program `expr`, which evaluates a reverse Polish expression from the command line, where each operator or operand is a separate argument. For example,

```
expr 2 3 4 + *
```

evaluates 2 × (3+4). □